

Trabalho – Gerenciamento de memória

Objetivo: Analisar a taxa de falhas da TLB.

1) Obtendo os traces:

- Instale o valgrind

Utilizando o arquivo bz2

- Acesse o site <https://valgrind.org/downloads/> e faça o download do arquivo fonte (<https://sourceware.org/pub/valgrind/valgrind-3.27.1.tar.bz2>), ou a versão mais recente.
- Descompacte o arquivo
- cd valgrind-3.27.1
- ./configure
- make
- sudo make install

- Gerando os traces utilizando a ferramenta lackey. Obs: A ferramenta possui algumas limitações, acesse o arquivo lk_main.c para uma descrição mais detalhada.

a) Dado o programa main.c

```
int main(){  
    printf("Teste");  
}
```

gcc main.c -o main

valgrind --log-file=main.trace --tool=lackey --trace-mem=yes ./main

O trace tem o seguinte formato:

```
==21229== Lackey, an example Valgrind tool  
==21229== Copyright (C) 2002-2024, and GNU GPL'd, by Nicholas Nethercote  
et al.  
==21229== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright  
info  
==21229== Command: ./main  
==21229== Parent PID: 381  
==21229==  
I 04020290,3  
I 04020293,5  
S 1fff0001a8,8  
I 04021030,4  
I 04021034,1
```

Cabeçalho:
desconsiderar

Cada linha representa um acesso à memória

Formato: Operação Endereço, NumeroDeBytes

I= leitura de instrução no endereço 0x04020290 de 3 bytes

Operações sobre dados: S= store, L=load,

```

L 1fff000128,8
I 0402104c,2
I 0402104e,7
I 04021055,7
M 0403ae0e,1
... (trecho omitido)
...
I 04937c2f,2
==21229==
==21229== Counted 1 call to main()
==21229==
==21229== Jccs:
==21229==   total:          22,455
==21229==   taken:          9,088 (40%)
==21229==
==21229== Executed:
==21229==   SBs entered:    22,247
==21229==   SBs completed: 15,015
==21229==   guest instrs:   130,440
==21229==   IRStmts:         798,852
==21229==
==21229== Ratios:
==21229==   guest instrs : SB entered   = 58 : 10
==21229==           IRStmts : SB entered = 359 : 10
==21229==           IRStmts : guest instr = 61 : 10
==21229==
==21229== Exit code:      0

```

Informações finais:
desconsiderar

Gerando traces maiores, por exemplo no comando abaixo será gerado trace da execução do gcc para compilar o código main.c

```
valgrind --log-file=gcc.trace --tool=lackey --trace-mem=yes gcc main.c
```

- 2) Gerar o *reference string*, considerando que cada página tem tamanho de 4096 bytes e o espaço de endereçamento possui 48 bits, logo são necessários 12 bits para o deslocamento e 36 bits para o número da página.

```

I 04020290,3    página 04020
I 04020293,5    página 04020
S 1fff0001a8,8  página 1fff000
I 04021030,4    página 04021
I 04021034,1    página 04021

```

Reference string gerado: 04020, 1fff000, 04021

- 3) Calcular a taxa de compressão obtida com a representação do *reference string*.
- 4) Desenvolver um simulador de TLB. A ferramenta a ser desenvolvida no trabalho deverá calcular o número de falhas na TLB, tendo como entrada a *reference string* da execução de uma aplicação, considerando diferentes

tamanhos de TLB(P. exemplo 4, 6, 8, 10 entradas). **Caso seja necessário a substituição de entradas na TLB, a equipe deverá comparar dois algoritmos: Random e o LRU.**

Cada equipe deverá escolher 2 aplicações para gerar os traces e fazer a comparação entre os diferentes algoritmos de substituição de páginas (preferencialmente códigos desenvolvidos pelos próprios membros da equipe). Pelo menos um dos traces deverá ter mais de 2 milhões de acessos à memória (o *reference string* pode ser menor).

O que entregar: artigo, código fonte do simulador e das aplicações utilizadas para gerar o trace. (**ATENÇÃO, NÃO ENVIE OS TRACES!!!!!!**).

O formato do artigo está disponível em:

<https://www.ieee.org/conferences/publishing/templates.html> (não é necessário abstract)

Data de entrega: 26/08/2026 Código fonte e artigo na tarefa teams até as 8h.

Apresentação: a partir de 26/08/2026 (ordem a ser definida em sala de aula).